

Common Table Expressions (CTEs)

En las últimas dos clases estuvimos trabajando con subqueries en distintas posiciones: en el WHERE, en el FROM y en el SELECT. Ya viste que son herramientas poderosas. Pero también viste que cuando una consulta necesita varias subqueries, las cosas se empiezan a complicar. Metes una subquery dentro de otra, y esa dentro de otra más, y de pronto tienes tres o cuatro niveles de anidación que hay que leer de adentro hacia afuera para entender qué está pasando. Funciona, sí. Pero si vuelves a esa consulta seis meses después, vas a perder un buen rato descifrando qué hace cada pedazo.

Las CTEs — o Common Table Expressions — son la solución a ese problema. Y honestamente, de todas las formas de escribir subqueries, esta es la más profesional y la que vas a encontrar en código de producción serio.

El problema con las subqueries anidadas

Antes de ver cómo funcionan las CTEs, vale la pena sentir el problema que resuelven. Pensemos en esta pregunta de negocio: *"Identifica los tres mejores vendedores por país, mostrando solo los países donde el volumen total de ventas supera cierto umbral"*.

Para responder esto con subqueries necesitas tres pasos lógicos: primero, calcular las ventas por empleado y país. Segundo, asignarle un ranking a cada empleado dentro de su país. Y tercero, filtrar solo los países cuyo volumen total supere el umbral que te piden. Tres pasos claros.

El problema es que con subqueries anidadas, esos tres pasos quedan empotrados uno dentro del otro. La consulta se lee de adentro hacia afuera: primero tienes que encontrar la subquery más interna, entenderla, luego subir un nivel, entender cómo usa ese resultado, subir otro nivel más, y finalmente llegar al SELECT externo. En la clase lo vimos en pantalla: la consulta funciona perfectamente, pero entenderla requiere ir pelando capas como una cebolla. Y si alguien la tiene que modificar meses después — y ese alguien probablemente vas a ser tú — la experiencia no es agradable.

Ahora imagina que no son tres niveles sino cinco o siete. Eso pasa en proyectos reales, y es ahí donde las subqueries anidadas se convierten en lo que se llama

deuda técnica: código que funciona pero que nadie quiere tocar porque nadie lo entiende del todo.

CTEs: los mismos pasos, pero escritos como un proceso

Una CTE es, en esencia, una subquery con nombre. En vez de anidar consultas una dentro de otra, las defines al inicio de tu query, les pones un nombre descriptivo, y después las usas como si fueran tablas normales. La palabra clave es **WITH**.

Tomemos la misma pregunta de negocio de los tres vendedores top por país. Con CTEs, la consulta se organiza así:

- **Paso 1** — Defines una CTE llamada **VentasPorEmpleado** que calcula las ventas de cada empleado agrupadas por país. Eso queda guardado en memoria con ese nombre.
- **Paso 2** — Defines una segunda CTE llamada **ConRanking** que toma el resultado de **VentasPorEmpleado** y le agrega un número de ranking a cada empleado dentro de su país. Esta CTE referencia a la anterior por su nombre, como si fuera una tabla.
- **Paso 3** — Defines una tercera CTE llamada **PaísesRelevantes** que filtra solo los países cuyo volumen total de ventas supera el umbral.

Resultado final — Un **SELECT** simple que cruza **ConRanking** con **PaísesRelevantes**, filtra los que tienen ranking menor o igual a 3, y ordena.

El resultado es idéntico al de las subqueries anidadas. Pero la diferencia en legibilidad es enorme. La consulta se lee de arriba hacia abajo, como una receta: haz esto primero, después esto, después esto, y al final dame el resultado. Cada paso tiene un nombre que te dice qué hace. No hay que pelar capas ni leer de adentro hacia afuera.

Las reglas del juego

Hay algunas cosas que necesitas saber sobre cómo se comportan las CTEs en SQL Server para usarlas bien.

Todo bloque de CTEs empieza con la palabra **WITH** y termina cuando llega la query principal — el **SELECT** final (o un **INSERT**, **UPDATE** o **DELETE**, aunque eso es menos común). Puedes definir varias CTEs en un mismo bloque separándolas con comas, y

cada CTE puede referenciar a las que se definieron antes que ella. Eso es lo que permite encadenar pasos lógicos.

Un punto importante: la CTE existe solamente durante la ejecución de esa query. No se guarda en la base de datos, no la puedes llamar desde otra consulta, no persiste. Es temporal por naturaleza. Si necesitas algo que se comporte como una CTE pero que cualquier persona pueda usar sin escribir SQL, eso es una Vista, y la vamos a ver en el próximo módulo.

También hay un matiz técnico que vale la pena mencionar: en SQL Server, las CTEs no se "materializan" automáticamente. Eso significa que el motor puede decidir ejecutar la lógica de una CTE varias veces si la referencias en más de un lugar. En la mayoría de los casos eso no es un problema, pero si notas que el rendimiento se degrada en consultas muy pesadas, la alternativa es volcar el resultado intermedio en una tabla temporal (`#TempTable`). Eso lo veremos más adelante en el curso.

CTEs recursivas: cuando la tabla se referencia a sí misma

Hay un caso especial de CTE que abre una puerta que con subqueries normales simplemente no puedes abrir: la **CTE recursiva**. Es una CTE que se referencia a sí misma, y sirve para recorrer datos jerárquicos — estructuras donde un registro apunta a otro registro de la misma tabla.

El ejemplo clásico es un organigrama. Imagina que en la tabla `Empleados` cada persona tiene un campo `SupervisorID` que indica quién es su jefe. El jefe de todos (el CEO, digamos) tiene ese campo en NULL porque no reporta a nadie. A partir de ahí, cada empleado apunta a su supervisor, que a su vez apunta al suyo, y así se forma un árbol.

Una CTE recursiva recorre ese árbol en dos partes. La primera es el **caso base**: encuentra los empleados que no tienen supervisor (los que están en la cima). La segunda es el **caso recursivo**: busca a todos los empleados cuyo supervisor ya fue encontrado en el paso anterior, y se repite hasta que no quedan más niveles por recorrer.

El resultado es una tabla donde cada empleado aparece con su nivel en la jerarquía y el camino completo desde el jefe hasta él. Con subqueries normales, resolver esto requeriría saber de antemano cuántos niveles tiene el organigrama y escribir un JOIN por cada nivel. Con una CTE recursiva, SQL lo resuelve solo, sin importar la profundidad.

Una precaución: si los datos tienen algún error y se forma un ciclo (el empleado A reporta a B, B reporta a C, y C reporta a A), la recursión entra en un bucle infinito. Para protegerte de eso, SQL Server tiene la opción `MAXRECURSION`, que le pone un límite de profundidad. Si la recursión supera ese límite, la consulta se detiene y te avisa.

¿Por qué importa tanto la legibilidad?

Puede parecer que esto es solo una cuestión de estética — que da igual si la consulta está escrita con subqueries anidadas o con CTEs si el resultado es el mismo. Pero en la práctica, la legibilidad tiene consecuencias reales. Código que no se entiende no se mantiene, no se revisa bien y acumula errores silenciosos. Cuando trabajas en equipo, alguien más va a tener que leer tu consulta. Cuando trabajas solo, ese "alguien más" es tu yo del futuro que ya no recuerda qué hacía cada pieza.

La regla que plantea el instructor es simple: si tu consulta tiene más de dos o tres pasos lógicos claros, usa CTEs. Tu código queda más limpio, más fácil de depurar y mucho más fácil de modificar cuando cambien los requerimientos — que siempre cambian.

Desafío de cierre

En los recursos de la clase encontrarás una consulta escrita con subqueries anidadas. Tu tarea es reescribirla usando CTEs. Identifica los pasos lógicos que hay dentro de las subqueries, dale un nombre claro a cada uno, y arma el bloque WITH paso por paso. Comparte ambas versiones en los comentarios de la clase y explica qué hace cada CTE.